

Programmatic Large-Scale Minecraft Terrain Architecture

A Technical Blueprint for Automated 10,000 × 10,000 Block Continents

Author: Ashenfall Core Engineering Group | **Version:** 2.5.0-Enterprise | **Target:** Minecraft 1.21.1+ (World Machine / Gaea to WorldPainter)

EXECUTIVE SUMMARY:

Constructing massive photorealistic Minecraft continents (10,000 × 10,000 blocks, covering 100 km² and 100,000 surface columns) cannot be accomplished through manual brush painting. Hand-painting at this scale causes unnatural repetition, concentric brush rings, broken hydrological flow, and unsustainable human labor overhead.

This document outlines the complete three-stage programmatic architecture: (1) **Procedural Geological Simulation** (spline-guided instance scattering, thermal scree weathering, and priority-flood river routing); (2) **Ecological Probability Transformation** (multi-criteria evaluation and blue-noise spatial dithering); and (3) **Headless WorldPainter Script Automation** (declarative config files driving JSR-223 Rhino ECMAScript execution).

Document Structure & PDF Outline

Chapter	Title & Core Technical Topics	Domain
1	Executive Summary & The Paradigm Shift Voxel scaling limits, declarative simulation vs. manual painting, pipeline overview.	System Overview
2	Stage 1: Procedural Geological Synthesis Spline instance scattering, Frenet-Serret frames, thermal weathering, priority-flood rivers.	Geomorphology
3	Stage 2: Ecological Probability & Distribution Continuous MCE, treeline/snowline response curves, blue-noise spatial dithering.	Spatial Statistics
4	Stage 3: Headless WorldPainter Automation Declarative YAML schemas, Rhino JSR-223 scripting host, wpscript CLI integration.	Compiler & CLI
5	Production Code Architecture & Reference Modules End-to-end Python/Java code for scattering, weathering, dithering, and JSR223 synthesis.	Reference Source
6	Tool Comparison & Production Execution Guide World Machine vs. Gaea vs. Custom Python; troubleshooting common artifacts.	Implementation

1. Executive Summary & The Paradigm Shift

When building at 10,000 × 10,000 scale, traditional terraforming tools fail due to fundamental human and software scaling constraints. A human artist operating a 200-block diameter brush in WorldPainter would have to perform over 50,000 distinct brush strokes to cover the continent once, inevitably producing unnatural circular ripples and inconsistent topography.

The programmatic paradigm treats terrain as a **directed acyclic graph (DAG) of physical transformations**. Artistic intent is expressed through high-level guide vectors (mountain spine splines, river sources, coast boundaries), while mathematical simulations execute low-level erosion, slope texturing, and object placement.

Pipeline Stage	Input Artifact	Applied Mathematics / Logic	Output Artifact
1. Geology	Guide Splines & Boundaries	Cubic splines, arête instances, stream power law, thermal slumping	16-Bit Lossless Heightmap
2. Distribution	Heightmap & Gradients	Slope derivative $ \nabla H $, ecological MCE, blue-noise spatial dithering	Binary Layer Masks (Populate, Frost)
3. Automation	Heightmap + Masks + Config	JSR-223 Rhino ECMAScript compiler, wpscript CLI automation	Minecraft Anvil Region (.mca)

2. Stage 1: Procedural Geological Synthesis

2.1 Instance Scattering for Mountain Ridgelines

Standard Perlin noise produces isotropic mounds without tectonic structure. Real mountain ranges consist of continuous structural axes formed by tectonic uplift. The system utilizes **instance scattering** along parametric guide curves:

- **Structural Fault Spline $C(t)$:** A parametric Hermite or Bézier curve $(x(t), z(t))$ with unit tangent vector $T(t)$ and normal $N(t)$.
- **Arête Primitive:** $h_{ridge}(u, v) = H_0 \cdot \exp(-|v| / \sigma) \cdot (1 - (u/L)^2) \cdot (1 + \eta \cdot \text{Noise}(u, v))$.
- **Polynomial Smooth-Max Blending:** $\text{smax}(a, b, k) = (a + b + \sqrt{(a - b)^2 + k}) / 2$. This avoids both flat plateau truncation and razor-sharp derivative seams, creating organic mountain passes (cols).

2.2 Thermal Weathering & Talus Slumping

Rock faces cannot exceed the natural **angle of repose** ($\theta_c \approx 35^\circ\text{--}40^\circ$). Material exceeding this angle breaks away and deposits at the cliff foot as concave parabolic talus aprons. The finite-difference cellular model displaces excess mass downhill along the direction of steepest descent $(-\nabla H / \|\nabla H\|)$, creating natural scree slopes and rugged cliff faceting.

2.3 Flow Restructuring & Hydrological River Routing

Standard procedural noise generates hollow pits that trap water. The system resolves this using a three-tier hydrological engine:

1. **Priority-Flood (Barnes / Wang-Liu):** Raises landlocked depressions to spillway levels or carves breach lines, guaranteeing monotonic descent to sea level ($H(p_{i+1}) \leq H(p_i)$).
2. **D-Infinity Flow Accumulation:** Calculates the upstream catchment area $A(x, z)$ for every cell based on aspect angles.

3. **Stream Power Law Carving:** Valley incision follows $\Delta H = -K \cdot A^m \cdot \|\nabla H\|^n$ ($m \approx 0.5$, $n \approx 1.0$), cutting deep V-shaped mountain canyons and wide meandering lowland floodplains.

3. Stage 2: Ecological Probability & Distribution

3.1 The Fundamental Challenge: Continuous Fields vs. Discrete Voxels

Simulation software computes continuous floating-point fields (height $H \in [-64, 320]$, slope $\theta \in [0^\circ, 90^\circ]$, moisture $M \in [0, 1]$). Minecraft, however, requires discrete integer block IDs and exact point coordinates for trees, rocks, and foliage.

3.2 Multi-Criteria Evaluation (MCE) & Ecological Suitability

For each feature, a suitability function $P_{\text{feature}}(x, z)$ evaluates multiple environmental parameters:

- **Slope-Aware Topsoil Law:** Slopes $< 35^\circ$ receive 100% deep soil; slopes $> 45^\circ$ are stripped to bare stone scree.
 - **Climatic Treeline:** Sigmoidal probability drop between $Y=180$ and $Y=225$ blocks; above $Y=225$, tree probability is 0.
 - **Glacial Snowline:** Deep snow accumulates above $Y=210$ on gentle slopes, but is shed on steep cliff faces.

3.3 Blue Noise Spatial Dithering (Void-and-Cluster)

Thresholding continuous probability against standard white noise causes unnatural clustering and bare voids. The system utilizes **blue noise dithering** (void-and-cluster matrix), which maximizes spatial distance between placed points. The resulting binary mask distributes foliage, fallen logs, and boulders with natural biological randomness and zero grid artifacts.

4. Stage 3: Headless WorldPainter Automation

4.1 Declarative Configuration Architecture

Rather than manually clicking in WorldPainter, all world dimensions, build limits, strata bands, and mask mappings are specified in a single declarative configuration file (`terrain_config.yaml`).

```
world:
  name: "Ashenfall_Continent"
  width: 10000; height: 10000; min_y: -64; max_y: 320; sea_level: 62
inputs:
  heightmap: "exports/height_16bit.png"
  populate_mask: "exports/mask_populate.png"
  frost_mask: "exports/mask_frost.png"
strata:
  - range: [-64, 61]; terrain: "Gravel / Sand"
  - range: [62, 130]; terrain: "Grass / Bare Dirt"; slope_limit: [0, 35]
  - range: [131, 190]; terrain: "Podzol / Coarse"
  - range: [191, 250]; terrain: "Stone / Cobblestone"
  - range: [251, 320]; terrain: "Deep Snow"
```

4.2 Automated JSR-223 Rhino Script Compiler

A Python compiler translates `terrain_config.yaml` into a deterministic ECMAScript file executed by WorldPainter's internal scripting engine (`wpscript`):

```
var heightMap = wp.getHeightMap().fromFile("exports/height_16bit.png").go();
var world = wp.createWorld().fromHeightMap(heightMap).scale(100)
    .fromLevels(0, 65535).toLevels(-64, 320).withWaterLevel(62)
    .withLowerBuildLimit(-64).withUpperBuildLimit(320).go();
var popMask = wp.getHeightMap().fromFile("exports/mask_populate.png").go();
var popLayer = wp.getLayer().withName("Populate").go();
wp.applyHeightMap(popMask).toWorld(world).applyToLayer(popLayer).fromLevels(128, 255).toLevel(1).go();
wp.exportWorld(world).toDirectory(".minecraft/saves/Ashenfall").go();
```

5. Production Software Architecture & Reference Modules

The complete pipeline is implemented as five decoupled Python and JavaScript modules:

- 1. ``instance_scatterer.py``: Evaluates 3D cubic splines and composites exponential arête ridges using smooth-max.
- 2. ``thermal_weathering.py``: Vectorized finite-difference cellular automata for slope slumping.
- 3. ``hydraulic_router.py``: Monotonic downhill stream power carver.
- 4. ``blue_noise_dither.py``: Converts continuous suitability matrices into un-clumped binary placement masks.
- 5. ``jsr223_compiler.py``: Emits verified WorldPainter scripts for headless CLI execution.

6. Practical Tooling Comparison

Feature	World Machine	QuadSpinner Gaea	Custom Python / C++
Architecture	Node-based CPU DAG	Node-based GPU DAG	Pure Scripting / SIMD
Free Limit	512 × 512 Basic	1024 × 1024 Community	Unlimited (RAM bound)
Hydrology	River Reach Device	Hydraulic Erosion	Priority-Flood + Stream Power
Automation	CLI Batch Builds	Enterprise Automation	100% Native Headless

Document Reference: Complete architectural markdown specification is available in the repository at `docs/design/PROGRAMMATIC_LARGE_SCALE_TERRAIN_SYSTEM_SPECIFICATION.md`.